机器人信息发布

1、程序功能说明

小车启动底盘驱动之后，会发布超声波、四路巡线等传感器模块数据，可以在docker中，运行命令去查询这些信息，也可以发布速度、蜂鸣器、RGB灯条等传感器的控制数据。

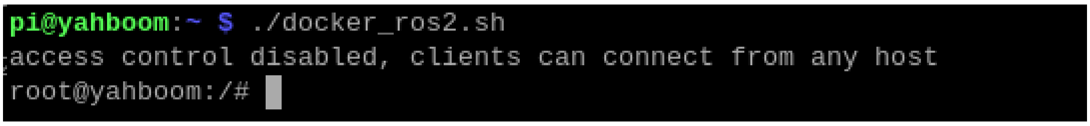
2、查询小车信息

2.1、进入小车docker

打开一个终端输入以下指令进入docker，

```
./docker_ros2.sh
```

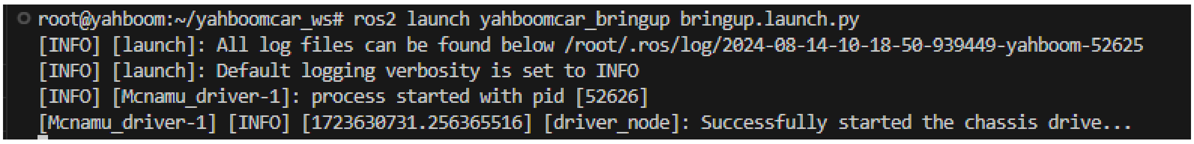
出现以下界面就是进入docker成功，现在即可通过指令控制小车，



```
pi@yahboom:~ $ ./docker_ros2.sh
access control disabled, clients can connect from any host
root@yahboom:/#
```

```
ros2 launch yahboomcar_bringup bringup.launch.py
```

启动底盘驱动，后如下图所示，



```
root@yahboom:~/yahboomcar_ws# ros2 launch yahboomcar_bringup bringup.launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2024-08-14-10-18-50-939449-yahboom-52625
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [Mcnamu_driver-1]: process started with pid [52626]
[Mcnamu_driver-1] [INFO] [1723630731.256365516] [driver_node]: Successfully started the chassis drive...
```

再新开一个终端，进入同一个docker，以下的 da8c4f47020a 修改成实际终端显示的ID

```
docker ps
```

```
docker exec -it da8c4f47020a  /bin/bash
```

```
pi@yahboom:~ $ docker ps
CONTAINER ID   IMAGE                                    COMMAND                  CREATED         STATUS
    PORTS     NAMES
da8c4f47020a   yahboomtechnology/ros-humble:0.0.4   "/ros_entrypoint.sh …"   8 hours ago    Up 45 minute
s             festive_payne
pi@yahboom:~ $ docker exec -it da8c4f47020a /bin/bash
root@yahboom:/#
```

输入以下指令查询节点，

```
ros2 node list
```

```
root@yahboom:~# ros2 node list
/driver_node
```

然后输入以下指令查询该节点发布/订阅了哪些话题，

```
ros2 node info /driver_node
```

```
root@yahboom:~# ros2 node info /driver_node
/driver_node
  Subscribers:
    /buzzer: std_msgs/msg/Bool
    /cmd_vel: geometry_msgs/msg/Twist
    /rgblight: std_msgs/msg/Int32MultiArray
    /servo: yahboomcar_msgs/msg/ServoControl
  Publishers:
    /line_sensor: std_msgs/msg/Int32MultiArray
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
    /ultrasonic: std_msgs/msg/Float32
  Service Servers:
    /driver_node/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /driver_node/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /driver_node/get_parameters: rcl_interfaces/srv/GetParameters
    /driver_node/list_parameters: rcl_interfaces/srv/ListParameters
    /driver_node/set_parameters: rcl_interfaces/srv/SetParameters
    /driver_node/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:
```

可以看出订阅的话题有，

/buzzer：蜂鸣器控制

/cmd_vel：小车速度控制

/rgblight：RGB灯条控制

/servo：s1舵机云台控制，s2舵机云台控制

发布的话题有，

/ultrasonic：超声波模块数据

/line_sensor：四路巡线模块数据

我们也可以通过查询话题命令，终端输入，

```
ros2 topic list
```

```
root@yahboom:~# ros2 topic list
/buzzer
/cmd_vel
/line_sensor
/parameter_events
/rgblight
/rosout
/servo
/ultrasonic
```

2.3、查询话题数据

查询超声波数据，

```
ros2 topic echo /ultrasonic
```

```
root@yahboom:~# ros2 topic echo /ultrasonic
data: 190.39999389648438
---
data: 190.39999389648438
---
data: 467.5
---
data: 469.20001220703125
---
data: 467.5
---
data: 467.5
---
data: 467.5
---
data: 469.20001220703125
---
```

查询四路巡线模块数据，

```
ros2 topic echo /line_sensor
```

```
root@yahboom:~# ros2 topic echo /line_sensor
layout:
  dim: []
  data_offset: 0
data:
- 0
- 0
- 1
- 0
---
layout:
  dim: []
  data_offset: 0
data:
- 0
- 0
- 1
- 0
---
layout:
  dim: []
  data_offset: 0
data:
- 0
- 0
```

3、发布小车控制信息

3.1、控制蜂鸣器

首先查询以下蜂鸣器话题的相关信息，终端输入，

```
ros2 topic info /buzzer
```

```
root@yahboom:~# ros2 topic info /buzzer
Type: std_msgs/msg/Bool
Publisher count: 0
Subscription count: 1
root@yahboom:~#
```

得知数据类型是std_msgs/msg/Bool。然后输入以下指令打开蜂鸣器，终端输入，

```
ros2 topic pub /buzzer std_msgs/msg/Bool "data: 1"
```

```
root@yahboom:~# ros2 topic pub /buzzer std_msgs/msg/Bool "data: 1"
publisher: beginning loop
publishing #1: std_msgs.msg.Bool(data=True)
```

输入以下指令关闭蜂鸣器，终端输入，

```
ros2 topic pub /buzzer std_msgs/msg/Bool "data: 0"
```

```
^Croot@yahboom:~# ros2 topic pub /buzzer std_msgs/msg/Bool "data: 0"
publisher: beginning loop
publishing #1: std_msgs.msg.Bool(data=False)
```

3.2、发布速度控制信息

我们假设发布小车以线速度0.1的速度，终端输入，

```
#有效值范围
#linear:     x[-1,1] y[-1,1] z
#angular:    x       y       z[-3,3]
```

```
ros2 topic pub -1 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

```
root@yahboom:~# ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.1, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0
))
```

停车则输入，

```
ros2 topic pub -1 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

3.3、控制RGB灯条

我们假设发布RGB值255,0,0，终端输入

```
ros2 topic pub -1 /rgblight std_msgs/msg/Int32MultiArray "data: [255, 0, 0]"
```

```
root@yahboom:~# ros2 topic pub -1 /rgblight std_msgs/msg/Int32MultiArray "data: [255, 0, 0]"
publisher: beginning loop
publishing #1: std_msgs.msg.Int32MultiArray(layout=std_msgs.msg.MultiArrayLayout(dim=[], data_offset=0), data=[255, 0, 0])
```

关灯则输入

```
ros2 topic pub -1 /rgblight std_msgs/msg/Int32MultiArray "data: [0, 0, 0]"
```

3.4、控制云台舵机

这里需要注意的是s1舵机的范围是[0,180]，s2舵机的范围是[0,110]，超过范围的值，舵机不转动。

我们假设控制s1舵机转动到 90度，s2舵机转动到 25度，则终端输入，

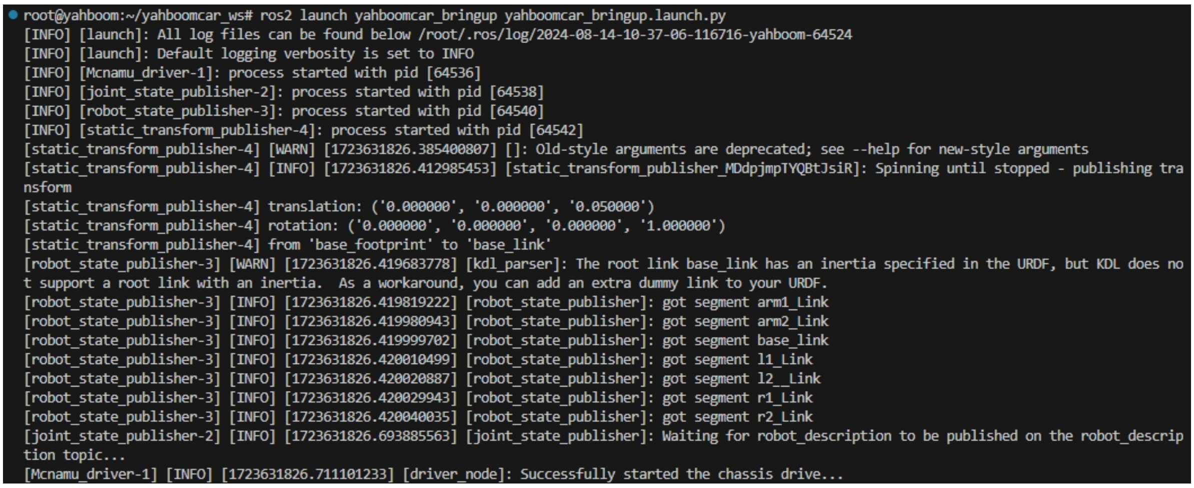
```
ros2 topic pub -1 /servo yahboomcar_msgs/msg/ServoControl "{'servo_s1': 90,
'servo_s2': 25}"
```

3.5、完整底盘驱动

3.5.1、启动程序

```
ros2 launch yahboomcar_bringup yahboomcar_bringup.launch.py
```

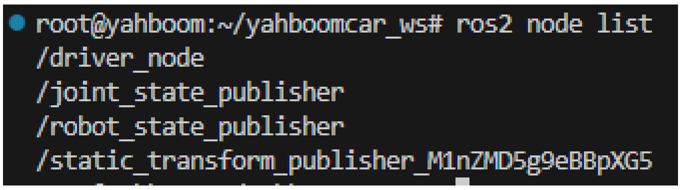
启动底盘驱动，后如下图所示，



```
root@yahboom:~/yahboomcar_ws# ros2 launch yahboomcar_bringup yahboomcar_bringup.launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2024-08-14-10-37-06-116716-yahboom-64524
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [Mcnamu_driver-1]: process started with pid [64536]
[INFO] [joint_state_publisher-2]: process started with pid [64538]
[INFO] [robot_state_publisher-3]: process started with pid [64540]
[INFO] [static_transform_publisher-4]: process started with pid [64542]
[static_transform_publisher-4] [WARN] [1723631826.385400807] []: Old-style arguments are deprecated; see --help for new-style arguments
[static_transform_publisher-4] [INFO] [1723631826.412985453] [static_transform_publisher_MDdpjmpTYQBtJsiR]: Spinning until stopped - publishing tra
nsform
[static_transform_publisher-4] translation: ('0.000000', '0.000000', '0.050000')
[static_transform_publisher-4] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-4] from 'base_footprint' to 'base_link'
[robot_state_publisher-3] [WARN] [1723631826.419683778] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does no
t support a root link with an inertia.  As a workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-3] [INFO] [1723631826.419819222] [robot_state_publisher]: got segment arm1_Link
[robot_state_publisher-3] [INFO] [1723631826.419980943] [robot_state_publisher]: got segment arm2_Link
[robot_state_publisher-3] [INFO] [1723631826.419999702] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1723631826.420010499] [robot_state_publisher]: got segment l1_Link
[robot_state_publisher-3] [INFO] [1723631826.420020887] [robot_state_publisher]: got segment l2__Link
[robot_state_publisher-3] [INFO] [1723631826.420029943] [robot_state_publisher]: got segment r1_Link
[robot_state_publisher-3] [INFO] [1723631826.420040035] [robot_state_publisher]: got segment r2_Link
[joint_state_publisher-2] [INFO] [1723631826.693885563] [joint_state_publisher]: Waiting for robot_description to be published on the robot_descrip
tion topic...
[Mcnamu_driver-1] [INFO] [1723631826.711101233] [driver_node]: Successfully started the chassis drive...
```

终端输入以下指令查询节点，

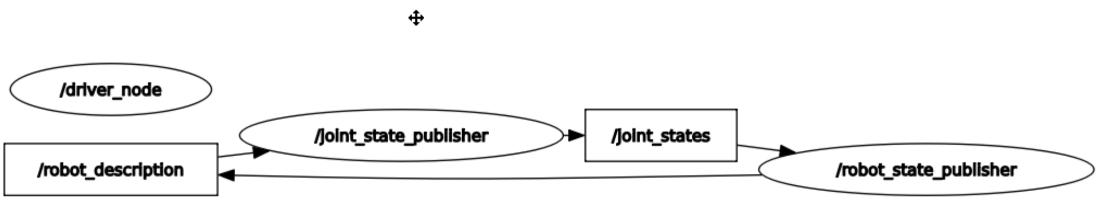
```
ros2 node list
```



```
root@yahboom:~/yahboomcar_ws# ros2 node list
/driver_node
/joint_state_publisher
/robot_state_publisher
/static_transform_publisher_M1nZMD5g9eBBpXG5
```

输入以下指令，查看节点间的通讯图，

```
ros2 run rqt_graph rqt_graph
```



输入命令查看节点的相关信息

```
ros2 node info /driver_node
```

```
root@yahboom:~/yahboomcar_ws# ros2 node info /driver_node
/driver_node
  Subscribers:
    /buzzer: std_msgs/msg/Bool
    /cmd_vel: geometry_msgs/msg/Twist
    /rgblight: std_msgs/msg/Int32MultiArray
    /servo: yahboomcar_msgs/msg/ServoControl
  Publishers:
    /line_sensor: std_msgs/msg/Int32MultiArray
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
    /ultrasonic: std_msgs/msg/Float32
  Service Servers:
    /driver_node/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /driver_node/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /driver_node/get_parameters: rcl_interfaces/srv/GetParameters
    /driver_node/list_parameters: rcl_interfaces/srv/ListParameters
    /driver_node/set_parameters: rcl_interfaces/srv/SetParameters
    /driver_node/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:
```

3.5.2、解析launch文件

```python
from ament_index_python.packages import get_package_share_path

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition, UnlessCondition
from launch.substitutions import Command, LaunchConfiguration

from launch_ros.actions import Node
from launch_ros.parameter_descriptions import ParameterValue

import os
from ament_index_python.packages import get_package_share_directory

from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource

def generate_launch_description():
    # 配置参数

    description_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('yahboomcar_description'), 'launch'),
         '/description_launch.py'])
    )

    driver_node = Node(
        package='yahboomcar_bringup',
        executable='Mcnamu_driver',
    )

    # 返回LaunchDescription对象
    return LaunchDescription([
```

```
        driver_node,
        description_launch
    ])
```

该launch文件启动了以下几个节点：

- driver_node：底盘驱动数据节点；
- description_launch：加载URDF模型。